

■ COMPLETE SCENARIO GUIDE



Git & GitHub

Real-World Scenarios

Basic Setup → Collaboration → Conflicts → Advanced Workflows

Replaces & expands the README scenario · Krish6115/GitxGithub

How to Use This Guide

Each scenario below is a self-contained real-world situation with the exact Git commands to handle it. Start from Scenario 1 (first-time setup) and work through to Scenario 10 (advanced workflows). Every command is explained so you know *why*, not just *what*.

#	Scenario	Key Commands
01	First-Time Setup & Identity	git config, ssh-keygen
02	Upload Your First File	init / clone, add, commit, push
03	Working with Branches	branch, checkout, merge
04	Collaborating via Forking	fork, upstream, pull request
05	Handling Merge Conflicts	conflict markers, manual resolve
06	Undoing Mistakes	restore, reset, revert, reflog
07	Keeping Your Fork in Sync	fetch, rebase, reset --hard
08	Team Workflow (Feature Branch)	PR, review, squash merge
09	Tagging & Releasing Versions	tag, push --tags, gh release
10	Advanced: Cherry-pick & Bisect	cherry-pick, bisect, blame

Scenario 01: First-Time Setup

Configure Git identity + SSH key on a new machine

■ Situation

You just got a new laptop. You need Git to know who you are, and you want to push to GitHub without typing your password every time.

This is a one-time setup per machine. Without the config step, Git will not let you commit.

1 Set your name and email (used in every commit)

```
● ● ● # Global config — applies to all repos on this machine
git config --global user.name "Krish6115"
git config --global user.email "your-email@gmail.com"

# Verify:
git config --global --list
```

2 Set default branch name to main

```
● ● ● # Modern default — avoids "master" naming
git config --global init.defaultBranch main
```

3 Generate SSH key and add to GitHub

```
● ● ● # Setup SSH key — do this once per machine
ssh-keygen -t ed25519 -C "your-email@gmail.com"
# Press Enter for default path, set a passphrase

eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# Copy your public key — paste this into GitHub:
cat ~/.ssh/id_ed25519.pub
```

■ ■ GitHub: Settings → SSH and GPG Keys → New SSH Key → paste the output of the cat command above.

4 Test the SSH connection

```
● ● ● # Confirm SSH works before first push
ssh -T git@github.com
# Expected: Hi Krish6115! You have successfully authenticated.
```

■ Once SSH is set up, you never need a password or token again on this machine.

Scenario 02: Upload Your First File

The complete flow from zero to pushed

■ Situation

GitHub Username: Krish6115
Repo: LeetCode | Folder: LinkedList
File to Upload: InsertLinkedList.java
Branch: main

Method A — Clone then Add (repo already exists on GitHub):

1 Clone the repo once

```
# Clone once — never again for this repo
git clone git@github.com:Krish6115/LeetCode.git
cd LeetCode
```

2 Move your file into the right folder

```
# Or drag-drop into the folder in VS Code
# On Mac/Linux:
mv /path/to/InsertLinkedList.java LinkedList/

# On Windows CMD:
move C:\path\InsertLinkedList.java LinkedList\
```

3 Stage the file

```
# Stage = tell Git which files to include in next commit
git add LinkedList/InsertLinkedList.java

# OR stage everything at once:
git add .

# Check what is staged:
git status
```

4 Commit with a message

```
● ● ● # Message should say WHAT changed and WHY
git commit -m "Add InsertLinkedList.java to LinkedList folder"
```

5 Push to GitHub

```
● ● ● # Push local commits to remote
git push origin main

# If rejected ("Updates were rejected"):
git pull origin main --rebase
git push origin main
```

Method B — Init fresh local project (no repo on GitHub yet):

```
● ● ● # Start fresh locally and connect to GitHub
mkdir MyProject && cd MyProject
git init

# Create your first file:
echo "# My Project" > README.md

git add .
git commit -m "Initial commit"

# Connect to a new GitHub repo (create repo on GitHub first):
git remote add origin git@github.com:Krish6115/MyProject.git
git push -u origin main
```

■ 'git push -u origin main' sets upstream so future pushes only need 'git push'.

For every new file after initial setup — just repeat these 3:

```
● ● ● # The 3-command daily loop
git add path/to/YourFile.java
git commit -m "Describe what this file does"
git push
```

Scenario 03: Working with Branches

Develop features without breaking main

■ Situation

You are adding a "user authentication" feature to your project.
You do NOT want to put broken code on main until it is ready.
You will create a branch, work on it, then merge back to main.

1 Create and switch to a feature branch

```
● ● ● # Branch names: use / to organize, e.g. feature/, fix/, hotfix/
# Old style (still works):
git branch feature/auth
git checkout feature/auth

# Modern one-liner:
git checkout -b feature/auth

# Even newer (Git 2.23+):
git switch -c feature/auth
```

2 Work and commit on the branch

```
● ● ● # Multiple commits on your branch is fine — squash later if needed
# ...make changes to files...

git add .
git commit -m "Add JWT authentication middleware"

git add .
git commit -m "Add login endpoint"

# Push branch to GitHub (first time):
git push -u origin feature/auth
```

3 Merge feature branch into main

```
● ● ● # Always pull main before merging to catch conflicts early
# Switch back to main:
git checkout main

# Pull latest main first:
git pull origin main

# Merge your branch:
git merge feature/auth

# Push merged main:
git push origin main
```

4 Delete branch after merge

```
● ● ● # Clean up — keep the branch list tidy
# Delete local branch:
git branch -d feature/auth

# Delete remote branch:
git push origin --delete feature/auth
```

■ **Rule: one branch = one feature/fix. Never commit directly to main.**

■■ 'git branch -d' fails if branch is not merged. Use '-D' to force delete (careful!).

Scenario 04: Contributing via Fork

Contribute to any public repo on GitHub

■ Situation

You want to contribute to someone else's public repo (e.g., an open source project).
You don't have write access. You fork it, make changes, and open a Pull Request.

1 Fork on GitHub and clone your fork

```
● ● ● # You now have a local copy of YOUR fork
# Click "Fork" on GitHub — creates your copy at:
# https://github.com/Krish6115/original-repo

# Clone YOUR fork:
git clone git@github.com:Krish6115/original-repo.git
cd original-repo
```

2 Add the original repo as "upstream"

```
● ● ● # origin = your fork, upstream = the original
git remote add upstream git@github.com:OriginalOwner/original-repo.git

# Verify both remotes:
git remote -v
# origin    git@github.com:Krish6115/original-repo.git (your fork)
# upstream  git@github.com:OriginalOwner/original-repo.git (source)
```

3 Create a branch and make changes

```
● ● ● # Always work on a branch — never on main of your fork
git checkout -b fix/login-null-check

# ...make your changes...

git add .
git commit -m "Fix null check in login function"
```

4 Push and open Pull Request

● ● ● # PR goes from YOUR fork/branch → ORIGINAL repo/main
`git push origin fix/login-null-check`

Then on GitHub.com:

Go to your fork → "Compare & pull request"

OR use GitHub CLI:

`gh pr create --base main --head Krish6115:fix/login-null-check`

Fill in title and description, select reviewers

■ ■ Keep PRs focused — one PR per bug fix or feature. Smaller PRs get reviewed faster.

■ Add 'Closes #42' in the PR description to automatically close an issue when the PR merges.

Scenario 05: Handling Merge Conflicts

When two people edit the same line

■ Situation

You and your teammate both edited `src/auth.js`.

You both pushed to different branches and now you try to merge.

Git cannot decide which version to keep — it asks you to decide.

1 Git tells you there is a conflict

```
● ● ● # When you see "CONFLICT" — do not panic
git merge feature/auth
# CONFLICT (content): Merge conflict in src/auth.js
# Automatic merge failed; fix conflicts and then commit.

# See all conflicted files:
git status
```

2 Open the conflicted file and look for markers

```
● ● ● # Everything between <<< and === is YOUR code; === to >>> is THEIRS
# The file will contain:
<<<<<< HEAD                (your current branch)
    const timeout = 3000; (your version)
=====
    const timeout = 5000; (incoming branch version)
>>>>>> feature/auth        (their branch name)
```

3 Resolve by editing the file manually

```
● ● ● # Edit in VS Code — it shows a nice UI for this
# Delete the conflict markers and choose what to keep.
# Option A — keep yours:
    const timeout = 3000;

# Option B — keep theirs:
    const timeout = 5000;

# Option C — combine both:
    const timeout = process.env.TIMEOUT || 5000;
```

4 Stage the resolved file and commit

```
● ● ● # After resolving all conflicts, always stage and commit
git add src/auth.js

# Commit to finish the merge:
git commit -m "Merge feature/auth: use configurable timeout"

# Or abort if you want to start over:
git merge --abort
```

■ VS Code shows a visual merge editor — click 'Accept Current', 'Accept Incoming', or 'Accept Both' per conflict.

■ ■ Never leave <<<<<< markers in your code. They will crash your application at runtime.

Scenario 06: Undoing Mistakes

Every type of undo — matched to the situation

■ Situation

You made a mistake. The question is: WHEN did it happen?

- (a) Before staging? (b) Staged but not committed?
(c) Committed locally? (d) Already pushed to remote?

When	Command	Effect
File edited, not staged	<code>git restore <file></code>	Discard edits, back to last commit
All files edited, not staged	<code>git restore .</code>	Discard ALL edits in working tree
File is staged	<code>git restore --staged <file></code>	Unstage file, keep your edits
Last commit, not pushed	<code>git reset HEAD~1 --soft</code>	Undo commit, keep changes staged
Last commit, not pushed	<code>git reset HEAD~1 --mixed</code>	Undo commit, keep changes unstaged
Last commit, not pushed	<code>git reset HEAD~1 --hard</code>	Undo commit AND discard changes
Already pushed to remote	<code>git revert <hash></code>	Create inverse commit (SAFE)
Accidentally deleted a branch	<code>git reflog + git branch <name> <reflog></code>	Recover any lost work

```
●●● # Revert is the safe choice after pushing
# Scenario: Committed and pushed wrong file to main
git log --oneline
# abc1234  Accidentally committed .env file

git revert abc1234 --no-edit
git push origin main

# Also remove the file from history (nuclear):
git filter-branch --force --index-filter \
  "git rm --cached --ignore-unmatch .env" HEAD
```

■ **git reset --hard is irreversible without reflog. Use git restore for file-level undo instead.**

Scenario 07: Keeping Fork in Sync

Stay up to date with the original repo

■ Situation

You forked a repo 2 weeks ago. The original has 30 new commits.

Your fork is now behind. You need to sync it before making more changes.

1 Fetch latest from upstream

```
● ● ● # Download all changes without applying them yet
git fetch --all --prune
# --prune removes tracking refs for deleted remote branches
```

2 Update your local main

```
● ● ● # Use reset --hard if main should exactly match upstream
git checkout main

# Method A – Reset (exact mirror of upstream):
git reset --hard upstream/main

# Method B – Rebase (replay your commits on top):
git rebase upstream/main

# Method C – Pull directly:
git pull upstream main
```

3 Push to your fork on GitHub

```
● ● ● # Update YOUR fork on GitHub too
git push origin main

# If you used reset --hard, you may need:
git push origin main --force-with-lease
```

4 Rebase feature branch on updated main

● ● ● # Keep your feature branch fresh against latest main

```
git checkout feature/my-feature
```

```
git rebase main
```

If conflicts appear during rebase:

1. Fix conflicts in files

2. git add <resolved-files>

3. git rebase --continue

To abort a rebase:

```
git rebase --abort
```

■ **Method 4 — Laziest:** click 'Sync fork' button on [GitHub.com](https://github.com) for a zero-command sync.

Scenario 08: Team Feature Branch Workflow

The professional GitHub team workflow

■ Situation

4-person Scrum team (sound familiar?). Each member works on a ticket.

Nobody touches main directly. All work goes through PRs with code review.

Goal: clean linear history on main.

1 Always start from latest main

```
● ● ● # Branching from latest main prevents conflicts later
git checkout main
git pull origin main
git checkout -b feature/JIRA-42-payment-integration
```

2 Commit often with meaningful messages

```
● ● ● # Conventional commits make changelogs and releases automatic
# Use conventional commits format:

git commit -m "feat: add Stripe payment gateway client"
git commit -m "fix: handle null response from payment API"
git commit -m "test: add unit tests for payment service"

# Conventional commit types:
# feat: (new feature)  fix: (bug fix)  docs: (documentation)
# test: (tests)  refactor: (code restructure)  chore: (maintenance)
```

3 Squash before PR (clean history)


```
● ● ● # Squash messy WIP commits into one clean commit before PR
# Interactive rebase to squash 5 commits into 1:
git rebase -i HEAD~5

# In the editor:
# pick a1b2c3d feat: add payment client
# squash d4e5f6g fix: handle null response
# squash h7i8j9k test: add tests
# squash l1m2n3o fix: typo in error msg
# squash p4q5r6s docs: update payment docs

# Save. Edit final commit message. Done.
```

4 Open PR and request review

```
● ● ● # PR = code review + discussion + CI checks, all in one place
git push origin feature/JIRA-42-payment-integration

gh pr create \
  --title "feat: Stripe payment gateway" \
  --body "Closes #42. Adds Stripe integration..." \
  --reviewer teammate1,teammate2
```

5 After approval — merge cleanly

```
● ● ● # Squash merge = one clean commit on main per feature
# On GitHub: use "Squash and Merge" for clean single commit
# OR via CLI:
gh pr merge 42 --squash

# Delete the branch:
gh pr merge 42 --squash --delete-branch
```

■ **Protect main:** Settings → Branches → Add rule → Require PR before merging + Require review.

Scenario 09: Tagging & Releasing Versions

Mark stable versions for production

■ Situation

Your project is stable. You want to mark it as version 1.0.0.

Teammates and users can always come back to this exact code.

GitHub should show a 'Release' with release notes.

1 Ensure main is clean and tested

```
● ● ● # Never tag untested code
git checkout main
git pull origin main
git log --oneline -5
# Confirm everything looks good
```

2 Create an annotated tag

```
● ● ● # Use semantic versioning: MAJOR.MINOR.PATCH
# Annotated tag (recommended — stores author, date, message):
git tag -a v1.0.0 -m "Version 1.0.0 — First stable release"

# See the tag details:
git show v1.0.0

# List all tags:
git tag
```

3 Push the tag to GitHub

```
● ● ● # Tags do not push automatically — you must do this explicitly
# Push one tag:
git push origin v1.0.0

# Push all local tags:
git push origin --tags
```

4 Create a GitHub Release

```
●●● # GitHub Release = tag + changelog + downloadable assets
# Via CLI:

gh release create v1.0.0 \
  --title "v1.0.0 – First Stable Release" \
  --notes "## What is new■- Feature A■- Bug fixes■- Performance improvements"

# Attach a build artifact (jar, zip, etc.):
gh release create v1.0.0 ./build/app.jar --title "v1.0.0"
```

Semantic Versioning Quick Reference:

Version Type	Example	When to Use
Major (BREAKING change)	v1.0.0 → v2.0.0	API changed, incompatible with old
Minor (new feature)	v1.0.0 → v1.1.0	Added functionality, backward compatible
Patch (bug fix)	v1.0.0 → v1.0.1	Bug fix, no new features
Pre-release	v1.0.0-alpha.1	Early testing versions

Scenario 10: Advanced: Cherry-pick & Bisect

Precision tools for complex situations

Cherry-pick Scenario:

■ Situation

A critical hotfix was committed on the 'dev' branch (commit abc1234).
You cannot merge all of dev into main yet — only this one fix is needed.
Cherry-pick that single commit onto main.

```
● ● ● # Cherry-pick just abc1234, leaving xyz5678 on dev
git log --oneline dev
# abc1234 hotfix: prevent SQL injection in search
# xyz5678 WIP: new dashboard (not ready)
# ...

git checkout main
git cherry-pick abc1234

# Confirm the fix is now on main:
git log --oneline -3
```

■ 'git cherry-pick abc1234..def5678' picks a range — excludes abc1234, includes everything after it up to def5678.

Bisect Scenario:

■ Situation

Sometime between last week and today, a bug appeared.
You have 200 commits. Manually testing each would take hours.
git bisect finds the exact culprit commit in ~8 steps.

```
●●● # Binary search: 200 commits → found in 8 steps
# Start bisect:
git bisect start

# Current commit is broken:
git bisect bad

# Tag a known-good commit (e.g., last week):
git bisect good v1.0.0

# Git checks out a middle commit.
# Run your test. If bug exists:
git bisect bad
# If no bug:
git bisect good

# Repeat ~7-8 times. Git prints:
# a3f4e2d is the first bad commit

# Exit bisect mode:
git bisect reset
```

Blame Scenario:

■ Situation

Line 47 of payment.js is causing a bug but you don't know when it was added.
git blame tells you the commit, author, and date for every line.

```
●●● # Find the exact commit that introduced a specific line of code
git blame -L 40,55 src/payment.js

# a1b2c3d4 (Krish 2025-01-20)    const amount = req.body.amount;
# e5f6g7h8 (Alice 2025-02-01)  if (amount <= 0) throw Error();

# See the full commit that changed line 47:
git show e5f6g7h8
```

Scenario : Master Quick Reference

All 10 scenarios summarized

Scenario	Key Commands	Remember
01 Setup	git config --global; ssh-keygen	Do once per machine
02 First Push	add → commit → push	The 3-step loop
03 Branch	checkout -b; merge; branch -d	1 branch = 1 feature
04 Fork	remote add upstream; PR	origin=yours, upstream=theirs
05 Conflict	fix markers; add; commit	Never leave <<< in code
06 Undo	restore / reset / revert / reflog	Pushed? Use revert.
07 Sync Fork	fetch --all; reset --hard upstream/main	Then push origin main
08 Team PR	rebase -i; squash; PR; merge	Review before merge
09 Release	tag -a; push --tags; gh release create	Use semantic versioning
10 Advanced	cherry-pick <hash>; bisect start	Precision > brute force

The Golden Rules

- Never commit directly to main
- One branch per feature/fix — keeps history clean
- Commit early and often — small commits are easier to revert
- Write meaningful commit messages — your future self will thank you
- Pushed to shared branch? Use git revert, not git reset
- Pull before you push — always sync first
- Use SSH keys — never type your GitHub password